

Jagiellonian University
Department of Theoretical Computer Science

Jakub Janór

**Implementation of Algorithms Based
on Planar Separators for Planar
Graphs**

Bachelor's Thesis

Supervisor: dr hab. inż. Krzysztof Turowski

June 2025

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Background	3
1.3	Definitions and important lemmas	3
1.3.1	Approximation algorithms	4
2	Planar Separator Theorem	5
2.1	Planar Separator Algorithm	5
2.2	Divide and Conquer Approach	11
3	Applications of planar separator theorem	16
3.1	Matching algorithm	16
3.1.1	Algorithm	16
3.1.2	Correctness	18
3.1.3	Time complexity	19
3.2	Vertex cover algorithm	20

Chapter 1

Introduction

1.1 Motivation

The planar separator theorem is a fundamental result in graph theory that states that any planar graph can be separated into two roughly equal parts by removing a small number of vertices. This theorem has numerous applications in various fields, including computer science, engineering, and mathematics.

1.2 Background

This thesis deals with algorithms based on planar separators. Let's define what a separator is at first then.

Definition 1.1. *A set $S \subseteq V(G)$ is a separator if induced graph $G^* = G - S$ has at least two connected components $A, B \subseteq G^*$.*

One of the first times this concept was brought to a broader audience was in [7], where Jordan proved that every free tree T has a separator of size 1 (a single vertex), whose removal partitions T into two or more connected components each of size at most $n/2$.

Through the years, more and more classes of graphs started to show some interesting properties regarding vertex separators, so a general theorem was assembled, which is called a $f(n)$ -separator theorem:

Theorem 1.2.1. *Let S be a class of graphs closed under a subgraph relation (i.e. if $G_1 \in S$ and G_2 is a subgraph of G_1 , then $G_2 \in S$). There exist constants $\alpha < 1$, $\beta > 0$ such that if G is any n -vertex graph in S , the vertices of G can be partitioned into three sets A, B, C such that no edge connects any vertex in A and B , neither A nor B have more than αn vertices and C contains no more than $\beta f(n)$ vertices*

Such theorem regarding planar graphs, where $f(n) = O(\sqrt{n})$, $\alpha = 2/3$, $\beta = 1$ is called planar separator theorem. First breakthrough in that area was in 1951 [11], when Ungar proved a slightly weaker theorem with $f(n) = O(\sqrt{n} \log^{3/2} n)$. A small improvement to that was later achieved in 1973 by George [5] where he proved that any grid graph of size n has a $O(\sqrt{n})$ separator. However, the first to prove planar separator theorem in the full form were Lipton and Tarjan in 1977 [8], where it was proven that for each planar graph a separator of size at most $2\sqrt{2}\sqrt{n}$ exists. In many articles later after Lipton and Tarjan, the constant β was reduced, however its lower bound is $(\sqrt{4\pi\sqrt{3}})/3$ which was proven by Djidjev in [4]. Djidjev established this absolute limit not by looking at flat grids, but by constructing worst-case planar graphs based on the triangulation of a sphere, proving mathematically that no algorithm can separate these dense spherical meshes with fewer vertices.

1.3 Definitions and important lemmas

Definition 1.2. *Graph G is a tuple (V, E) , where V is a vertex set and E is a set of edges*

Definition 1.3. *Let $G = (V, E)$ be a graph. We define a path P to be an induced*

1.3.1 Approximation algorithms

Chapter 2

Planar Separator Theorem

2.1 Planar Separator Algorithm

The main theorem, this whole thesis and all algorithms are based on is the planar separator theorem introduced by Lipton and Tarjan

Theorem 2.1.1 (Planar Separator Theorem [8]). *Let G be a planar graph with n vertices, with non-negative vertex costs summing to no more than one. Then we can partition vertices of G into three sets A , B , and C such that there exists no edge $u, v \in E(G)$ with $u \in A$ and $v \in B$, neither A nor B has more than $2/3$ cost, and C contains no more than $2\sqrt{2}\sqrt{n}$ vertices. Moreover, such a partition can be found in $O(n)$ time.*

To prove this theorem we will base on a few already proven facts on planar graphs as well as some lemmas, that we will introduce ourselves.

BFS layering

One of the most important parts of the proof of Theorem 2.1.1 is the concepts of layers which are levels of a BFS spanning tree.

Definition 2.1. *Let G be a connected planar graph and $t \in V(G)$ be a vertex that we will call the root of a tree. A BFS spanning tree is a tree T rooted in t such that $\forall_{v \in V(G)} d_T(t, v) = d_G(t, v)$.*

Using this spanning tree we can partition the vertex set V into disjoint sets that we will call layers (or levels interchangeably). So we can define it as $L_i = \{u \in V : d(t, u) = i\}$ ($L_0 = \{t\}$). Moreover we denote number of vertices on i -th level as $L(i) = |L_i|$. Such a layering has two important properties that will be very useful in the future.

Lemma 2.1.2. *If $(u, v) \in E$ and $u \in L_j, v \in L_i$, then $|i - j| \leq 1$*

The proof for this lemma is very simple by following the construction of BFS spanning tree during BFS traversal. The other property is also very intuitive and follows directly from Lemma 2.1.2.

Lemma 2.1.3. *For $l \in \{1, \dots, r - 1\}$ removal of L_l separates G into two disconnected components $\bigcup_{i=0}^{l-1} L_i$ and $\bigcup_{i=l+1}^r L_i$*

Theorem 2.1.4. *Let C be any closed curve in the plane. Removal of C divides the plane into exactly two connected regions: "inside" and "outside" of C .*

Lemma 2.1.5. *Let G be any planar graph. Shrinking any connected subgraph of G to a single vertex preserves planarity.*

Fundamental cycle

Definition 2.2. *Let $G = (V, E)$ be a graph and let T be a spanning tree of G . For any non-tree edge $e = (u, v) \in E \setminus E(T)$, the unique simple cycle consisting of e and the unique path in T connecting u and v is called the **fundamental cycle** defined by e .*

Lemma 2.1.6. *Let G be a planar graph with n vertices, with non-negative vertex costs summing to no more than one. Suppose that G has a spanning tree T with radius r . Vertices of G can be partitioned into three sets A, B, C such that A and B are disjoint, costs of A and B do not exceed $2/3$ and C consists of at most $2r + 1$ vertices, one of the vertices being the root of T .*

Proof. We shall see that if G has a vertex v with cost at least $1/3$, the lemma is true. Let C be the path from v to the root of T . We can see that it has less than $2r + 1$ vertices and contains the root. Moreover its cost is at least $1/3$ so if we assign the rest of vertices of G to A and let $B = \emptyset$ then the lemma is true. Thus let's assume that G does not have a vertex with cost at least $1/3$. Let's now embed G in plane using any algorithm (you can use one from this article [6]), and make that graph maximal. Now each one of the non-tree edges (those added when making G maximal as well) creates a cycle C with tree edges, its length being at most $2r + 1$ if it contains the root of T and $2r - 1$ otherwise. From 2.1.4 we know that C splits the plane into two parts the *inside* and *outside*. If we can prove that there exists a cycle C such that neither side contains vertices whose total cost exceeds $2/3$, then the lemma is proven. Let (x, z) be a non-tree edge that minimizes the maximum cost of either sides of the cycle C' that it closes. If there are some

more edges like that, pick the one with the smallest number of faces on the maximized side. Let's say without loss of generality, that the *inside* has the cost at least as high as the *outside*. If that cost does not exceed $2/3$ then the lemma is true. Otherwise let's consider a triangle F_T inside C' such that (x, z) is one of its boundaries. Let y be the third vertex of F_T . Now we have to do case analysis based on properties of the edges of F_T :

- if both $(x, y), (y, z)$ lie on the cycle then F_T is the cycle, which contradicts with the fact that inside has a positive cost, so it has to have vertices
- if one of $(x, y), (y, z)$ (assume without loss of generality it is (x, y)) lies on the cycle, then (y, z) is a non-tree edge that defines a new cycle C'' with the same cost, but one less face than C' which contradicts with choice of (x, z) .
- if none of $(x, y), (y, z)$, we have to make another case analysis looking at all possible configurations of those two edges being a tree edge. All those configurations lead to a contradiction (a complete proof is to be found in [8]).

As we can see, all of the cases lead to contradiction, so the cost inside C' has to be at most $2/3$. $\square$

Layering

Lemma 2.1.7. *Let G be a planar graph with n vertices, with non-negative vertex costs summing to no more than one. Suppose that vertices are partitioned into layers using a BFS spanning tree T rooted in t and with radius r . Let $r + 1$ be an additional level with no vertices. Given any two levels l_1, l_2 such that levels $0, \dots, l_1 - 1$ have total cost not exceeding $2/3$ as well as levels $l_2 + 1, \dots, r + 1$, it is possible to find a partition of G into sets A, B, C , such that neither A nor B have cost exceeding $2/3$, there is no edge connecting any vertex from A with any vertex from B and C has at most $L(l_1) + L(l_2) + \max\{0, 2(l_2 - l_1 - 1)\}$ vertices.*

Proof. Let's assume that $l_1 \geq l_2$. Then we can select $A = \bigcup_{i=0}^{l_1-1} L_i$, $B = \bigcup_{i=l_1+1}^{r+1} L_i$ and $C = L_{l_1}$. We can see that the lemma is true, as costs of A and C do not exceed their postulated costs. And because $l_1 \geq l_2$, then $B \subseteq \bigcup_{i=l_2+1}^{r+1} L_i$ so its cost does not exceed $2/3$.

Thus suppose $l_1 < l_2$. Let's define sets: $D_1 = \bigcup_{i=0}^{l_1-1} L_i$, $D_2 = \bigcup_{i=l_1+1}^{l_2-1} L_i$, $D_3 = \bigcup_{i=l_2+1}^{r+1} L_i$, all of which may be empty. We can see that they are disjoint by

Lemma 2.1.3. The only set that can have cost exceeding $2/3$ is D_2 (by assumptions of Lemma 2.1.7). If cost of D_2 is less or equal $2/3$, then the lemma is true, as we can set the most costly part be A , B other two parts and $C = L_{l_1} \cup L_{l_2}$.

Let's then suppose that the cost of D_2 is greater than $2/3$. Delete all vertices on levels l_2 and above, and shrink levels $0, \dots, l_1$ into one vertex t' with cost 0. From Lemma 2.1.5 we know that shrinking vertices preserves planarity. We can now see that the new graph H has a spanning tree rooted in t' and radius $l_2 - l_1 - 1$. Apply Lemma 2.1.6 to H and let A_H, B_H, C_H be the resulting partition. Let A be the set among A_H and B_H that has greater cost and let C consist of all of the vertices in L_{l_1}, L_{l_2} in the original graph G as well as all of the vertices from $C_H - \{t'\}$. We can see that it works because by Lemma 2.1.6, the cost of A does not exceed $2/3$. Moreover because of our construction, C has the size of at most $L(l_1) + L(l_2) + 2(l_2 - l_1 - 1)$. However we can see that the cost of $A \cup C_H$ has to be at least $1/3$, as A was the set with greater cost among A_H, B_H . Hence the cost of B is bounded by $2/3$ as the cost of $V(G) - D_2$ is less than $1/3$. $\square$

Main theorem

Now we have all of the tools to tackle Theorem 2.1.1.

Proof. Assume at first that G is connected. At first we partition into layers using BFS layering getting a spanning tree T rooted in t with radius r . Additionally we define two empty boundary levels L_{-1} and L_{r+1} . Let us define l_1 as the number of the level such that the cost of vertices included in levels 0 through $l_1 - 1$ is strictly less than $1/2$ and the cost of vertices in levels 0 through l_1 be at least $1/2$. If no such level exists then the cost of vertices is less or equal than $1/2$, and such we can assign $A = V(G)$, $B = C = \emptyset$, thus the theorem is true. We shall define $k = \sum_{i=0}^{l_1} L(i)$. Now, we find two levels:

- l_0 such that $l_0 \leq l_1$ and $L(l_0) + 2(l_1 - l_0) \leq 2\sqrt{k}$
- l_2 such that $l_1 + 1 \leq l_2$ and $L(l_2) + 2(l_2 - l_1 - 1) \leq 2\sqrt{n - k}$

If l_0, l_2 exist then by Lemma 2.1.7 (as by definition of l_1 and thus l_0, l_2 conditions for it hold) we have a partition A, B, C and the cost of C does not exceed $2(\sqrt{k} + \sqrt{n - k})$. Now by employing Cauchy-Schwartz inequality:

$$(u_1v_1 + u_2v_2)^2 \leq (u_1^2 + u_2^2)(v_1^2 + v_2^2)$$

with $u^T = [2, 2]$ and $v^T = [\sqrt{k}, \sqrt{n - k}]$ we can deduce that the cost of C is bounded by $2\sqrt{2}\sqrt{n}$. Now we shall see that such levels l_0 and l_2 exist. Let's

suppose that l_0 does not exist. Then for $\forall_{i \leq l_1} L(i) \geq 2\sqrt{k} - 2(l_1 - i)$. As $L(0) = 1$ we shall have $1 \geq 2\sqrt{k} - 2l_1$ and thus $l_1 + 1/2 \geq \sqrt{k}$. If we take the floor of that we have that $l_1 \geq \lfloor k \rfloor$. And now by definition of k we have:

$$\begin{aligned} k &= \sum_{i=0}^{l_1} L(i) \stackrel{(1)}{\geq} \sum_{i=l_1-\sqrt{\lfloor k \rfloor}}^{l_1} L(i) \stackrel{(2)}{\geq} \sum_{i=l_1-\sqrt{\lfloor k \rfloor}}^{l_1} (2\sqrt{k} - 2(l_1 - i)) \\ &\stackrel{(3)}{\geq} (4\sqrt{k} - 2\sqrt{\lfloor k \rfloor})(\sqrt{\lfloor k \rfloor} + 1)/2 \geq \sqrt{\lfloor k \rfloor}(\sqrt{\lfloor k \rfloor} + 1) > k \end{aligned}$$

Inequality (1) comes from the fact that we can restrict the range over which we sum as $l_1 + 1/2 \geq \sqrt{k}$. Inequality (2) stems from the fact that $L(i) \geq 2\sqrt{k} - 2(l_1 - i)$. And inequality (3) comes from the arithmetic sum formula. And thus we can see a contradiction, so l_0 exists. An analogous, symmetric proof can be given for l_2 existence. This concludes proof for a connected graph.

If G is not connected, let $G_1, \dots, G_k$ be connected components of G . If no connected component has cost exceeding $1/3$, then let i be the minimum index such that the cost of $V(G_1), \dots, V(G_i)$ exceeds $1/3$. Then we can set $A = V(G_1) \cup \dots \cup V(G_i)$, $B = V(G_{i+1}) \cup \dots \cup V(G_k)$ and $C = \emptyset$ and find our partition. When there is some component with cost at least $1/3$ and not exceeding $2/3$ let's say G_j , then we can set $A = V(G_j)$, $B = V(G) \setminus V(G_j)$, $C = \emptyset$ and find our partition. If there is some component with cost at least $2/3$ then we can employ for that connected component an argument in the first section of this proof to find a partition A^*, B^*, C^* . We can then set $C = C^*$, A will be the vertex set among A^* and B^* that has the greater cost and B shall be the rest of the vertices in the graph. It is easy to see that this concludes our proof. $\square$

$O(n)$ Algorithm

Following the constructions used in order to proof Theorem 2.1.1, Lipton and Tarjan developed an $O(n)$ algorithm to find a partition described in it. On the input we get a planar graph G for example in adjacency lists representation

1. Find a planar embedding of G using an efficient $O(n)$ algorithm described in [6].
2. Find connected components of G and calculate costs of each one. If there is no cc with cost at least $2/3$, then follow the last part of the proof for Theorem 2.1.1. Otherwise, go to the next step.

3. Find a BFS spanning tree of the most costly component. Compute the level of a vertex and for each level, compute $L(l)$, so the number of vertices on l -th level.
4. Find l_1 such that the cost of levels $0, \dots, l_1 - 1$ does not exceed $1/2$ and the cost of levels $0, \dots, l_1$ is greater or equal that $1/2$. Now find the highest level $l_0 \leq l_1$ such that $L(l_0) + 2(l_1 - l_0) \leq 2\sqrt{k}$ and lowest level $l_2 \geq l_1 + 1$ such that $L(l_2) + 2(l_2 - l_1 - 1) \leq 2\sqrt{n - k}$.
5. Delete all vertices on level l_2 and above. Construct a new vertex x and shrink all the levels from 0 to l_0 to that one vertex, rewiring the edges on the boundary of l_0 level to x . (to see how it is exactly done you can refer to [8] or implementation of the algorithm in `PlanarSeparator.cpp`)
6. Construct a BFS spanning tree T_x rooted in x . Keep track for each vertex v of its parent in T_x and the total cost of descendants of v including v itself. Make the graph maximal by triangulation.
7. Choose any non-tree edge (v_1, w_1) and by following parent pointers find a corresponding cycle C_1 . Compute costs of either sides of the cycle and call the side with greater cost *inside*.
8. Find a cycle with inside cost not exceeding $2/3$ using Algorithm 1.
9. Use the cycle found in the previous step and levels found in step 4 to construct a vertex partition as described in proof for Lemma 2.1.7. Extend the partition found for the most costly connected component to entire graph like in the proof for Theorem 2.1.1.

Time complexity

It is easy to see that all the steps, but step 8, take $O(n)$ time. So all we have to do is prove this lemma:

Lemma 2.1.8. *Algorithm 1 has time complexity $O(n)$.*

Proof. In each step of the algorithm we remove at least one face of the planar graph, so the number of iterations is bound by $O(n)$. In each iteration we do some $O(1)$ work, work proportional to the length of path P_T and some work proportional to the number of edges scanned in cycles R_1, R_2 . We can see that if $e \in P_T$, then that computation regarding tree path is done only once, as that edge will in the next iteration be on the boundary of new cycle. So all the work regarding computations of P_T amortizes to $O(n)$. Regarding scanning, we can see that if we scanned k edges in each cycle before stopping

we did computations proportional to $O(k)$. But in the next iteration at least k edges will be outside the next cycles. So those computations amortize to $O(n)$ as well. $\square$

Algorithm 1 FUNDAMENTAL_CYCLE

```

1:  $i \leftarrow 1$ 
2:  $C_i \leftarrow C_1$ 
3: Let  $(v_i, w_i)$  be a non-tree edge completing  $C_i$ 
4: while cost inside  $C_i$  exceeds  $2/3$  do
5:   Let  $(v_i, y, w_i)$  be a triangle inside  $C_i$  with boundary edge  $(v_i, w_i)$ 
6:   if  $(v_i, y) \in E(T_x)$  or  $(w_i, y) \in E(T_x)$  then
7:     Let  $(v_{i+1}, w_{i+1})$  be a non-tree edge among  $(v_i, y), (w_i, y)$ 
8:     Let  $C_{i+1}$  be a cycle defined by  $(v_{i+1}, w_{i+1})$ 
9:     Compute cost inside  $C_{i+1}$ 
10:  else
11:    Find tree path  $P_T$  from  $y$  to  $C_i$  path
12:    Let  $z$  be the first vertex in  $P_T$  from  $y$  that is on  $C_i$ 
13:    Compute cost of all vertices on  $P_T \setminus \{z\}$ 
14:    We have two cycles  $R_1$  defined by  $(y, v_i)$  and  $R_2$  defined by  $(y, w_i)$ 
15:    Scan alternatively one edge in  $R_1$  and one in  $R_2$ 
16:    Stop scanning when one of those two cycles have been scanned
17:    Find the cost of both cycles by reusing cost of  $C_i$  and  $P_T \setminus \{z\}$ 
18:    Let  $(v_{i+1}, w_{i+1})$  be the edge among  $(v_i, y), (w_i, y)$  whose cycle has
    more cost inside
19:  end if
20: end while

```

2.2 Divide and Conquer Approach

We can recall the divide and conquer approach to solve problems [3]. We can divide the problem into smaller subproblems, solve each subproblem independently, and then combine the results to get the solution to the original problem.

Approximation algorithms for NP-complete problems

If we look at Theorem 2.1.1, we can see that it gives us a way to divide a planar graph into smaller parts efficiently. This is particularly useful for finding approximate solutions to NP-complete problems on planar graphs in polynomial time. Let's look at the following lemma.

Lemma 2.2.1. *Let G be a planar graph with n vertices, with non-negative vertex costs summing to no more than one, and let $0 < \epsilon < 1$. Then there exists some set C of $O(\sqrt{n/\epsilon})$ vertices such that removal of C from G leaves no connected component with cost more than ϵ . Moreover, such a set can be found in $O(n \log n)$ time.*

Proof. We will consider two cases in here:

If $\epsilon \leq 1/n$, we can simply take $C = V(G)$, which has $O(\sqrt{n/\epsilon})$ vertices and $G - C = \emptyset$, so it has no connected components with cost greater than ϵ trivially.

If $\epsilon > 1/n$, we introduce a simple algorithm:

Algorithm 2 Finding ϵ -planar separator

```

1:  $C \leftarrow \emptyset$ 
2: while  $G - C$  has a component with cost greater than  $\epsilon$  do
3:   Let  $K$  be such a connected component
4:   Apply Theorem 2.1.1 to  $K$  producing a partition  $A_1, B_1$ , and  $C_1$ 
5:    $C \leftarrow C \cup C_1$ 
6: end while
7: return  $C$ 

```

We can see that in each iteration of the while loop of the algorithm, we split the greatest component into two smaller ones, each with cost at most $2/3$ of the original component. Let $L : P(V) \rightarrow \mathbb{N}$ be a function that assigns a level to each component that arises during the execution of the algorithm. We can define it as follows: $L(K) = 0$ if K is the component of $G - C$ when the algorithm halts. Otherwise $L(K) = 1 + \max(L(A_i))$, where A_i is the component created by splitting K during the execution of the algorithm. With such a definition, we can see that $\forall_{A,B} L(A) = L(B) \implies A \cap B = \emptyset$, because all components that are produced in the same iteration of the while loop are disjoint by construction of the algorithm and we can see that if $A \subset B$, then $L(A) < L(B)$, because A is produced by splitting B . Moreover, we can see that each level one component has cost at least ϵ , because it is yet to be split by the algorithm. Thus for $i \geq 1$, each component has cost at least $(3/2)^{i-1}\epsilon$, as it has to be split $i - 1$ times for some component to get to level 1, when it will have cost at least ϵ . And by Theorem 2.1.1, each time its cost will be $2/3$ of cost before splitting. Now let's observe that because the cost of whole graph G is at most 1, then the total number of components at level i is at most $1/((3/2)^{i-1}\epsilon) = (2/3)^{i-1}/\epsilon$. In particular, the maximum level k of the component must satisfy

$$1 \leq (2/3)^{k-1}/\epsilon \leq (2/3)^{k-1}n$$

So we can derive from that $k \leq 1 + \log_{3/2} n$. We can then see that the height of the recursion tree of the algorithm is $O(\log n)$ and thus the algorithm will run in $O(n \log n)$ time.

Now we need to prove the bound on the size of C . Let $K_1, \dots, K_m$ be the components on some level $i \geq 1$ of sizes $n_1, \dots, n_m$ respectively. When splitting each of these components, we add at most $2\sqrt{2} \sum_{j=1}^m \sqrt{n_j}$ vertices to C by Theorem 2.1.1. We can recall now the bound m by the number of components at i -th level, namely: $m \leq (2/3)^{i-1}/\epsilon$. We trivially also have $\sum_{j=1}^m n_j \leq n$. To obtain the higher bound on $|C|$ we want to maximize the sum $\sum_{j=1}^m \sqrt{n_j}$ subject to constraint $\sum_{j=1}^m n_j \leq n$. For some fixed level m that sum is maximized by setting $n_j = n/m$. Now we can find a bound for the number of vertices added to C on each level:

$$2\sqrt{2} \sum_{j=1}^m \sqrt{n_j} \leq 2\sqrt{2} \sum_{j=1}^m \sqrt{n/m} = 2\sqrt{2} \sqrt{nm} \leq 2\sqrt{2} (2/3)^{(i-1)/2} \sqrt{n/\epsilon}$$

Now we can bound $|C|$:

$$|C| \leq \sqrt{n/\epsilon} \sum_{i=1}^{\infty} 2\sqrt{2} (2/3)^{(i-1)/2}$$

We know that the series converges to some constant as it is just a geometric series with $|q| < 1$, which means that $|C| = O(\sqrt{n/\epsilon})$ $\square$

By setting cost of each vertex in the graph to $1/n$ we have to following:

Corollary 2.2.1.1. *Let G be a planar graph with n vertices and let $0 < \epsilon < 1$. Then there exists some set C of $O(\sqrt{n/\epsilon})$ vertices such that removal of C from G leaves no connected component with more than ϵn vertices. Moreover, such a set can be found in $O(n \log n)$ time.*

Now we will introduce a class of problems described as **maximum induced subgraph problem** P on graph G with some graph property Q . The problem asks for a maximum set S of V that induces a subgraph with property Q [2]. Following just proved Lemma 2.2.1 we can deduce an algorithm (Algorithm 3, which we will call MISPP from now on). We can clearly observe that the algorithm halts and successfully solves the problem P .

Algorithm 3 MISIP algorithm

- 1: $0 < \epsilon < 1$
 - 2: Apply Corollary 2.2.1.1 to a given graph to find separator C and connected components of $G - C$ with maximum size ϵn
 - 3: In each component G_i of $G - C$ find a maximum set S_i inducing a subgraph with property Q by checking every subset of $V(G_i)$
 - 4: Return $S \leftarrow \bigcup S_i$
-

Now we will introduce a lemma, that shows an upper bound on time complexity of Algorithm 3.

Lemma 2.2.2. [2] *The approximation MISIP Algorithm (Algorithm 3) has time complexity $O(\max\{n \log n, n2^{\epsilon n}\})$ and relative error of $O(1/\sqrt{\epsilon n})$ if the following conditions are satisfied:*

1. *The subgraph of G induced by S satisfies property Q .*
2. *The error $|S^*| - |S|$ is bounded by the number of vertices of C , that is, $|S^*| - |S| = O(|C|)$, where S^* is a maximum vertex set inducing a subgraph of G with Q .*
3. *$|S^*|$ is a positive fraction of n .*
4. *Property Q is recognizable in linear time (that is, one can determine in linear time whether a graph satisfies Q or not).*

Proof. Time complexity

Algorithm *MISIP* has two main steps that affect the time complexity. The first step is simply $O(n \log n)$ by Corollary 2.2.1.1. In step two, for each connected component i we find S_i by checking every subset. Let $n_i = |V(G_i)|$, then we can find S_i in $O(n_i 2^{n_i})$ time (because from condition 4, we know that property Q is recognizable in linear time). So the total time is

$$O\left(\max\left\{\sum_{i=1}^n n_i 2^{n_i} \mid \sum_{i=1}^n n_i = n \text{ and } 0 \leq n \leq \epsilon n\right\}\right) = O\left(\frac{n}{\epsilon n} \epsilon n 2^{\epsilon n}\right) = O(n 2^{\epsilon n})$$

First equality stems from calculus and the fact that function $x2^x$ is strictly convex, so to maximize the value of the sum, we shall assign $n_i = \epsilon n$. We can see from both of those steps that the total time complexity has to be $O(\max\{n \log n, n 2^{\epsilon n}\})$.

Relative error

We shall define S^* as the maximum set of V that induces a subgraph satisfying property Q and S as the set produced by algorithm. We can see that condition 1 guarantees that we can compose a global solution S as a union of S_i , because each subgraph of G induced by S_i satisfies property Q . From condition 2 we shall see that $|S^*| - |S| = O(|C|) = O(\sqrt{n/\epsilon})$. Combining that with condition 3 that guarantees us $|S^*| = \Omega(n)$ and by performing some simple operations on asymptotic functions we get that:

$$\frac{|S^*| - |S|}{|S^*|} \leq c_1 \sqrt{\frac{n}{\epsilon}} \cdot \frac{c_2}{n} = c_1 c_2 \sqrt{\frac{1}{\epsilon n}} = O(1/\sqrt{\epsilon n})$$

Which concludes our proof. □

Chapter 3

Applications of planar separator theorem

3.1 Matching algorithm

3.1.1 Algorithm

In this section we will discuss an efficient algorithm for finding a maximum matching in a planar graph. The maximum matching problem asks for a maximum number of pairwise nonadjacent edges in a graph. Although there exists an exact algorithm to find a maximum matching for planar graphs in $O(n\sqrt{n})$ time, we can based on theorems recalled in Chapter 2 exhibit an approximation algorithm that runs in $O(n \log n)$ time and whose worst-case ratio is $1 - O(1/\sqrt{\log \log n})$. One can find it interesting why we cannot use Algorithm *MISP* directly and have worst-case ratio and running time guaranteed by Lemma 2.2.2. That is because the third property is not satisfied. The counter example is graph $K_{1,n-1}$ whose matching size is 1 regardless of the size of the graph, and that property demands that the size of the matching is a positive fraction of n . It is also really important to see that even though *MISP* algorithm was defined for problems with sets of vertices it easily extendable to sets of edges which we can employ in this section.

Algorithm 4 MATCHING procedure

```
1: procedure MATCHING
2:   REDUCE(G, S(G))
3:   return S(G)
4: end procedure
```

Algorithm 5 REDUCE procedure

```
1: procedure REDUCE( $G, s(G)$ )
2:    $N \leftarrow |V(G)|$ 
3:    $v \leftarrow \min_{u \in V(G)} \deg(u)$ 
4:   if  $N \leq \log \log n$  then
5:     Find exact matching  $S^*(G)$  using any reasonable algorithm
6:      $S(G) \leftarrow S^*(G)$ 
7:   else if  $\deg(v) < 3$  then
8:     if  $\deg(v) = 0$  then
9:        $G' \leftarrow G - v$ 
10:      REDUCE( $G', S(G')$ )
11:       $S(G) \leftarrow S(G')$ 
12:    else if  $\deg(v) = 1$  then
13:       $u \leftarrow$  vertex adjacent to  $v$ 
14:       $G' \leftarrow G - \{u, v\}$ 
15:      REDUCE( $G', S(G')$ )
16:       $S(G) \leftarrow S(G') \cup \{(v, u)\}$ 
17:    else if  $\deg(v) = 2$  then
18:       $u, w \leftarrow$  neighbors of  $v$ 
19:       $G' \leftarrow$  graph made from  $G$  by identifying  $v, u$  and  $w$ 
20:      REDUCE( $G', S(G')$ )
21:      if  $S(G')$  does not have an edge adjacent to  $u$  in  $G$  then
22:         $S(G) \leftarrow S(G') \cup \{(v, u)\}$ 
23:      else
24:         $S(G) \leftarrow S(G') \cup \{(v, w)\}$ 
25:      end if
26:    end if
27:  else
28:    Run Algorithm MISP with  $\epsilon = \log \log n / N$  on  $G$ 
29:     $S(G) \leftarrow$  result of algorithm
30:  end if
31: end procedure
```

Theorem 3.1.1. *The approximation algorithm MATCHING for the maximum matching problem on planar graphs has running time $O(n \log n)$ and worst-case ratio $1 - O(1/\sqrt{\log \log n})$.*

Proof. **3.1.2 Correctness**

We will start the proof to Theorem 3.1.1 by proving that it returns a matching with stated worst-case ratio. For that we will split the proof into two parts. First we will show that reductions performed by REDUCE procedure make the approximation error smaller than that of the graph before a reduction. And secondly, we will show that in base cases the algorithm returns the correct worst-case ratio. We will define the maximum matching of G as $S^*(G)$ and the matching produced by algorithm as $S(G)$. We will show that

$$\frac{|S(G)|}{|S^*(G)|} \geq 1 - O\left(\frac{1}{\sqrt{\log \log n}}\right)$$

Reductions

1. G has a vertex of degree 0. Let $G' = G - v$. Then we have that $S(G) = S(G')$, as there is no edge adjacent to v . For the same reason we have $S^*(G) = S^*(G')$, so the reduction does not affect the ratio.
2. G has a vertex of degree 1. Let u be the vertex adjacent to v in G and $G' = G - \{u, v\}$. If $S(G')$ is a matching produced by the algorithm then clearly $|S(G)| = |S(G')| + 1$ as we can add (u, v) to the matching. Also since $S^*(G)$ has exactly one edge adjacent to u then by removing it we can reduce the matching by at most one, so we have $S^*(G) \leq S^*(G') + 1$. Now we can derive:

$$\frac{|S(G)|}{|S^*(G)|} \geq \frac{|S(G')| + 1}{|S^*(G')| + 1} \geq \frac{|S(G')|}{|S^*(G')|}$$

where the last equality stems from properties of rational numbers.

3. G has a vertex of degree 2. Let u, w be vertices adjacent to v and G' be a graph obtained from G by identifying those three vertices into single vertex v' . Let $S(G) = S(G') \cup \{(u, v)\}$ if no edge adjacent to u in G was selected to $S(G')$. Otherwise let $S(G) = S(G') \cup (w, v)$. Clearly we have $|S(G)| = |S(G')| + 1$. Now if the matching $S^*(G)$ contains either (v, u) or (v, w) , let's say (v, u) then by removing it by contraction to v' , $S^*(G')$ is a correct matching of size at most one less than the latter. In the other case if both u and w were incident to edges in $S^*(G)$, then the matching $S^*(G')$ has to select one of them, so the property is conserved. In both cases we have $|S^*(G)| \leq |S^*(G')| + 1$. So again

$$\frac{|S(G)|}{|S^*(G)|} \geq \frac{|S(G')| + 1}{|S^*(G')| + 1} \geq \frac{|S(G')|}{|S^*(G')|}$$

We can see that all of the reductions create a graph if a worst-case ratio no worse than the original graph G .

Base cases

We have two base cases to discuss here:

1. If $N \leq \log \log n$, so we reduced our graph to a small fraction of the original one, we apply an exact matching algorithm, so $\frac{|S(G)|}{|S^*(G)|} = 1$ (if the denominator is 0, then we set this ratio to 1)
2. If $N \geq \log \log n$ and $\forall_{v \in V(G)} \deg(v) \geq 3$ we apply *MISP* algorithm. To prove its correctness we have to check conditions from Lemma 2.2.2:
 - (a) Satisfied, as $S(G)$ is a matching so a subgraph induced by it satisfies the property that no two edges can have a common vertex
 - (b) Let $S^*(G)$ be the maximum matching. By removing vertices of a separator C at most $|C|$ edges from the matching have incident vertices in C . So in the induced graph $G - C$ the maximum matching is of size at least $S^*(G) - |C|$. And this is also the size of $S(G)$ as we run the exact algorithm for matching here. So $|S(G)| \geq S^*(G) - |C|$, by rearranging we have $|S^*(G)| - |S(G)| \leq |C| = O(|C|)$, so the error is bound by size of C .
 - (c) Here we will use the theorem from [10] which guarantees that if G is planar and has minimum degree of at least 3, then $|S^*(G)| \geq N/3$, so the optimal solution is a positive fraction of N .
 - (d) We can easily check in linear time if the set found by the algorithm is the matching

We can see that all the conditions are satisfied, so by setting $\epsilon = \log \log n / N$ we have the desired worst-case error by Lemma 2.2.2.

3.1.3 Time complexity

At first we should discuss the time complexity of reductions. By keeping a graph as adjacency lists, so that each vertex not only has a previous reference and next reference but also the cross reference to other linked lists, we can perform a deletion in $O(\deg(v))$. And as each vertex is deleted only once, we can do that in amortized time $O(1)$. Another operation is identification of vertices which can be performed in total time $O(n \log n)$ [2]. To do that we keep an additional adjacency matrix and rewiring edges from the vertex of

a smaller degree to one of higher. As each vertex is present in $O(\log n)$ such rewirings, we get the needed time complexity. Now we can look at running time of base cases. If $N \leq \log \log n$ we use an exact algorithm to find the matching. If its time complexity is at most $O(n2^N)$, then it is enough for our desired time complexity. And if $N \geq \log \log n$, then from Lemma 2.2.2 we have guaranteed $O(\max\{N \log N, N2^{\epsilon N}\}) = O(N \log n) = O(n \log n)$. $\square$

3.2 Vertex cover algorithm

Another problem we will look at is Vertex Cover. In [2] we can find an algorithm very similar to the one for matching problem discussed in a previous section. However for VC another interesting approach may be found in [1] which will be shown in this section. This algorithm once again is based on approximation scheme that can be found in [9] and that was already discussed in Chapter 2. So we will get an approximation algorithm with running time $O(n \log n)$ and error $O(\sqrt{\log \log n})$.

Definition 3.1. Let $G = (V, E)$ be a simple planar graph. A rosary in G is a simple circuit of even length $2l$, denoted by $v_1 - v_2 - \dots - v_{2l} - v_1$, such that the degree of every even-indexed vertex is exactly two for all $i = 1, 2, \dots, l$.

Definition 3.2. Let $G = (V, E)$ be a simple planar graph. A vertex $v \in V$ of degree two is called a bead if it plays the role of an even-indexed vertex v_{2i} in some rosary in G .

Definition 3.3. Let $G = (V, E)$ be a simple planar graph. We will call it proper if the degree of all vertices is at least 2 and the set Y_B of all its beads satisfies $|Y_B| < 1/6|V|$.

In [1] authors prove a very useful theorem regarding proper graphs.

Theorem 3.2.1. Let $G = (V, E)$ be a simple planar graph with all vertices having degree at least 2. Then we have:

$$|VC^*| > \frac{1}{5}(|V| - |Y_B|)$$

If G is proper then by Theorem 3.2.1 we have $|V|/|VC^*| < 6$.

Bibliography

- [1] R. Bar-Yehuda and S. Even. “On Approximating a Vertex Cover for Planar Graphs”. In: *Proceedings of the Fourteenth Annual ACM Symposium on Theory of Computing - STOC '82*. San Francisco, California, United States: ACM Press, 1982, pp. 303–309. ISBN: 978-0-89791-070-5. DOI: 10.1145/800070.802205. (Visited on 05/01/2026).
- [2] Norishige Chiba, Takao Nishizeki, and Nobuji Saito. “Applications of the Lipton and Tarjan’s Planar Separator Theorem (Applied Combinatorial Theory and Algorithms)”. In: *RIMS Kokyuroku* 427 (1981), pp. 148–164. (Visited on 05/01/2026).
- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT press, 2022. (Visited on 06/05/2026).
- [4] Hristo Nicolov Djidjev. “On the problem of partitioning planar graphs”. In: *SIAM Journal on Algebraic Discrete Methods* 3.2 (1982), pp. 229–240.
- [5] J. Alan George. “Nested dissection of a regular finite element mesh”. In: *SIAM Journal on Numerical Analysis* 10.2 (1973), pp. 345–363.
- [6] John Hopcroft and Robert Tarjan. “Efficient planarity testing”. In: *Journal of the ACM (JACM)* 21.4 (1974), pp. 549–568.
- [7] Camille Jordan. “Sur les assemblages de lignes”. In: *Journal für die reine und angewandte Mathematik* 70 (1869), pp. 185–190.
- [8] Richard J. Lipton and Robert Endre Tarjan. “A Separator Theorem for Planar Graphs”. In: *SIAM Journal on Applied Mathematics* 36.2 (Apr. 1979), pp. 177–189. ISSN: 0036-1399, 1095-712X. DOI: 10.1137/0136016. (Visited on 05/01/2026).
- [9] Richard J. Lipton and Robert Endre Tarjan. “Applications of a Planar Separator Theorem”. In: *18th Annual Symposium on Foundations of Computer Science (Sfcs 1977)*. IEEE, 1977, pp. 162–170. (Visited on 05/01/2026).

- [10] Takao Nishizeki and Ilker Baybars. “Lower bounds on the cardinality of the maximum matchings of planar graphs”. In: *Discrete Mathematics* 28.3 (1979), pp. 255–267.
- [11] Peter Ungar. “A theorem on planar graphs”. In: *Journal of the London Mathematical Society* 1.4 (1951), pp. 256–262.